

Microsoft GraphRAG avec un Knowledge Graph RDF

Partie 1 - Reecriture technique fidele en francais

D'apres l'article de Ian Ormesher : Microsoft GraphRAG with an RDF Knowledge Graph - Part 1, publie sur Medium le 12 aout 2024.

Nature du document

Ce document n'est pas une reproduction integrale de l'article. C'est une reecriture technique fidele, en francais, construite uniquement a partir des elements verifies dans l'article Medium et dans le depot GitHub associe. Les interpretations sont signalees comme telles.

Objectif pratique

Cette premiere partie explique comment faire fonctionner Microsoft GraphRAG avec un serveur local compatible avec l'API OpenAI, en utilisant un modele de conversation et un modele d'embeddings disponibles localement. Le resultat attendu est la production des fichiers de sortie de GraphRAG, qui seront ensuite transformes en donnees RDF dans la partie 2.

Sources principales

- Article Medium : <https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-1-00a354afdb09>
- Depot GitHub associe : https://github.com/ianormy/msft_graphrag_blog

1. Intention de l'article

L'auteur part d'un constat simple : Microsoft GraphRAG est interessant pour construire un graphe de connaissances a partir d'un corpus de textes, mais son execution peut necessiter de nombreux appels vers un modele compatible OpenAI. Ces appels peuvent rapidement devenir couteux si l'on utilise un service externe payant. La demarche proposee consiste donc a experimenter GraphRAG en local, sans envoyer les donnees vers un site externe.

La partie 1 se concentre sur le montage de l'environnement local : serveur compatible OpenAI, modele de chat, modele d'embeddings, configuration de GraphRAG et generation des fichiers de sortie. Les parties suivantes utilisent ces sorties pour construire un Knowledge Graph RDF, puis pour interroger ce graphe avec SPARQL dans une demarche RAG.

2. Position de la partie 1 dans la serie

La serie est organisee en trois temps. La premiere partie configure GraphRAG pour utiliser un serveur local. La deuxieme recupere les sorties produites par GraphRAG et les charge dans un magasin RDF, en l'occurrence GraphDB. La troisieme utilise le Knowledge Graph ainsi constitue pour ameliorer les reponses RAG a l'aide de SPARQL 1.1.

Point important

La partie 1 ne cree pas encore le Knowledge Graph RDF final. Elle prepare les donnees en executant Microsoft GraphRAG. Le passage vers RDF, l'ontologie OWL, GraphDB et l'index vectoriel appartiennent a la partie 2.

3. Exigences pour le serveur local

L'auteur retient plusieurs criteres pour choisir le serveur local. Le serveur doit pouvoir utiliser des modeles au format GGUF, offrir des points d'accès pour le chat et pour les embeddings, être open source, fonctionner sur la machine locale et éviter l'envoi des données vers un service externe.

Ces criteres sont directement lies au fonctionnement de GraphRAG : l'outil a besoin d'un modele pour les operations de generation et d'un encodeur pour les embeddings. Il faut donc que le serveur expose deux capacites : une API de conversation et une API d'embeddings.

4. Pourquoi le format GGUF est central

Le choix du format GGUF est motive par la taille des modeles de langage. Les modeles complets et leurs poids peuvent occuper de nombreux gigaoctets. Le format GGUF permet de stocker ces modeles de facon plus compacte et de faciliter leur chargement en local. Dans la demarche de l'article, ce format est donc un critere d'exploitation pratique : il rend l' experimentation locale plus accessible.

L'article ne presente pas GGUF comme une garantie de qualite du modele. Il le presente plutot comme un format utile pour charger et executer des modeles locaux, notamment dans l'ecosysteme construit autour de llama.cpp.

5. Choix de l'outillage local

L'auteur observe que plusieurs serveurs locaux reposent sur llama.cpp. Il commence par considerer Ollama, puis s'oriente vers LM Studio. La raison donnee est pratique : dans la

version d'Ollama mentionnee, les fichiers GGUF sont importes puis convertis dans un format gere par Ollama, alors que l'auteur prefere conserver une visibilite directe sur les fichiers GGUF utilises.

LM Studio est donc choisi comme interface locale pour charger les modeles et exposer un serveur compatible avec les attentes de GraphRAG. Cette decision est un choix d'implementation de l'auteur, pas une comparaison exhaustive entre tous les serveurs locaux existants.

6. Modeles utilises dans l'experience

Dans l'article, deux modeles sont utilises : un modele de chat pour les interactions de type LLM et un modele d'embeddings pour encoder les textes. L'auteur indique avoir obtenu de bons resultats avec cette combinaison dans la suite de la serie.

Role	Modele cite dans l'article	Usage dans la chaine
Chat / generation	Meta-Llama-3.1-8B-Instruct-Q4_K_M.gguf	Modele local appele par GraphRAG pour les operations de generation.
Embeddings	bge-large-en-v1.5-f32.gguf	Modele local utilise pour produire les representations vectorielles.

Limite a ne pas depasser

L'article ne fournit pas de benchmark comparatif permettant d'affirmer que ces modeles sont les meilleurs. Il dit seulement qu'ils ont ete utilises dans cette experience et qu'ils ont donne de bons resultats pour la suite de la demonstration.

7. Reglage du contexte dans LM Studio

Une fois le modele charge dans LM Studio, l'auteur attire l'attention sur un parametre important : la longueur de contexte. La valeur par default observee dans LM Studio est jugee trop faible pour les besoins de GraphRAG. L'auteur indique avoir augmente cette valeur a 30 000 pour tenir compte des tailles de morceaux de texte que GraphRAG peut envoyer au modele.

Le point technique a retenir est que GraphRAG peut transmettre des blocs de texte relativement importants au modele. Si la fenetre de contexte du serveur local est trop petite, l'execution peut echouer ou tronquer des informations pertinentes. Le reglage de contexte doit donc etre coherent avec le comportement attendu de GraphRAG et avec les limites du modele charge.

8. Configuration de Microsoft GraphRAG

L'article suppose que le lecteur a deja suivi la procedure de demarrage de Microsoft GraphRAG et dispose d'un dossier de test, appele ragtest dans l'exemple. Deux fichiers doivent ensuite etre adaptes : le fichier .env et le fichier settings.yaml.

Dans .env, la cle API habituelle n'est pas une cle OpenAI reelle. Pour le serveur LM Studio local, l'auteur utilise simplement une valeur fictive adaptee a l'environnement local. Le role de ce parametre est donc de satisfaire la configuration attendue par GraphRAG, sans activer un service OpenAI externe.

Dans settings.yaml, plusieurs parametres sont modifies pour pointer vers le serveur local et vers les modeles charges dans LM Studio.

- Type d'API : utilisation de l'API de chat compatible OpenAI.
- Nom du modele LLM : correspondance avec le modele de chat charge localement.
- Nombre maximal de tokens : ajustement selon les capacites et les besoins de l'execution.
- API base : adresse locale affichee par LM Studio.
- Nombre de threads : l'auteur le fixe a 1, car LM Studio fonctionne sur la meme machine.
- Modele d'embeddings : correspondance avec le modele d'embeddings charge dans LM Studio.
- Taille des chunks : l'auteur revient a 300, en indiquant que cette valeur produit de meilleurs resultats dans son cas que la nouvelle valeur par defaut de 1200.

9. Execution de l'indexation GraphRAG

Lorsque LM Studio est pret et que les fichiers de configuration ont ete ajustes, l'indexation GraphRAG peut etre lancee depuis le repertoire de travail. L'article donne la commande suivante pour demarrer le traitement :

```
python -m graphrag.index --root ./ragtest
```

Cette commande lance la construction des sorties GraphRAG a partir du dossier racine indique. Le but de cette etape est de produire les fichiers qui serviront ensuite a alimenter le travail RDF de la partie 2.

10. Observation des prompts envoyes au modele

L'auteur signale une fonctionnalite utile de LM Studio : le mode verbose. En l'activant, on peut observer les prompts produits par GraphRAG et transmis au modele local. Cette observation est utile pour comprendre ce que GraphRAG demande au modele, diagnostiquer les problemes et verifier que l'environnement local recoit bien les appels attendus.

Cette etape n'est pas presentee comme obligatoire pour produire les sorties, mais comme un moyen de controle et de comprehension de la chaine de traitement.

11. Resultat attendu a la fin de la partie 1

A la fin de la partie 1, l'objectif n'est pas encore d'avoir un systeme RAG complet avec SPARQL. L'objectif est plus limite et plus fondamental : GraphRAG doit avoir termine son traitement et produit les fichiers de sortie correspondants. Ces fichiers, notamment au format Parquet dans la suite de la serie, deviennent la matiere premiere de la partie 2.

La logique globale est donc la suivante : GraphRAG extrait et structure des informations a partir du corpus; la partie 2 transforme ces resultats en representation RDF; la partie 3 utilise le Knowledge Graph RDF avec SPARQL pour soutenir la generation de reponses.

12. Ce que l'article ne pretend pas demontrer

Pour eviter les extrapolations, il faut distinguer ce que l'article montre de ce qu'il ne montre pas.

- Il ne fournit pas de comparaison systematique entre LM Studio, Ollama et d'autres serveurs locaux.

- Il ne donne pas de benchmark de performance materielle ni de recommandation officielle de GPU, CPU ou memoire.
- Il ne demontre pas encore l'interrogation SPARQL du graphe : cela appartient a la partie 3.
- Il ne remplace pas la documentation officielle de Microsoft GraphRAG; il illustre une configuration locale concrete.
- Il ne prouve pas que la taille de chunk 300 est universellement meilleure; il rapporte un choix de configuration favorable dans le contexte de l'auteur.

13. Lecture orientee GraphRAG semantique

Dans une perspective de GraphRAG fonde sur les technologies du Web semantique, cette partie 1 est surtout interessante parce qu'elle separe clairement deux niveaux. Le premier niveau est l'execution de Microsoft GraphRAG avec un LLM local et un encodeur local. Le second niveau, qui commence dans la partie 2, consiste a transformer les sorties de GraphRAG en donnees RDF interrogeables.

Cette separation est importante pour une architecture souveraine ou locale : le modele peut etre execute sans service externe, les sorties peuvent etre conservees localement, puis les structures extraites peuvent etre chargees dans un triple store RDF. L'approche ouvre donc une piste pour combiner LLM local, embeddings, ontologie OWL, RDF, GraphDB et SPARQL dans une chaine GraphRAG plus explicable et plus gouvernable.

Interpretation

L'article ne formule pas explicitement une doctrine de souverainete numerique. Toutefois, le fait de privilegier un serveur local, de ne pas transmettre les donnees a un site externe et de preparer un chargement RDF rend cette approche pertinente pour des architectures ou le controle local des donnees est important.

14. Resume operationnel

En termes operationnels, la partie 1 peut etre comprise comme une recette minimale pour faire tourner GraphRAG localement : choisir un serveur compatible OpenAI, charger un modele de chat et un modele d'embeddings, augmenter la longueur de contexte, adapter .env et settings.yaml, puis lancer l'indexation GraphRAG. Le livrable de cette etape est un ensemble de fichiers produits par GraphRAG, destines a etre repris dans la construction d'un Knowledge Graph RDF.

References

Les references ci-dessous correspondent aux sources utilisees pour cette reecriture. Elles sont indiquees sous forme d'URL afin que le document reste autonome.

- Ian Ormesher, Microsoft GraphRAG with an RDF Knowledge Graph - Part 1, Medium, 12 aout 2024 : <https://medium.com/@ianormy/microsoft-graphrag-with-an-rdf-knowledge-graph-part-1-00a354afdb09>
- Depot GitHub associe aux articles : https://github.com/ianormy/msft_graphrag_blog
- Documentation Microsoft GraphRAG : <https://microsoft.github.io/graphrag/>
- Projet Microsoft Research GraphRAG : <https://www.microsoft.com/en-us/research/project/graphrag/>
- llama.cpp : <https://github.com/ggerganov/llama.cpp>
- LM Studio : <https://lmstudio.ai/>
- Ollama : <https://ollama.com/>
- Specification GGUF : <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md>

Document genere pour usage de travail. Il s'agit d'une reecriture explicative, non d'une copie de l'article original.